

How the team works together

- **Communication**
 - Discord Channel
 - Specific text channels: ideas, general discussion, code help, urgent.
 - Messages should be responded to within 16 hours.
 - Pings should be answered immediately
 - In-person/video meetings
 - Meetings should discuss individual reports, topics from previous meetings, upcoming/ next step work, open discussion.
 - Shared work documents (word docs, github repo, etc.)
- **Settling disagreements**
 - Discuss with the group about possible actions.
 - Create team wide vote if necessary
 - Rotating position third-party mediator for disagreement between two members
- **Missed Expectations**
 - 20 Burpees **IN TOLLIVER**
 - Survey leftover members for skills that pertain to a lacking piece of work, then delegate based on some sort of randomization if more than one member can take over the task(s). (Least important)
 - 3 strike system - If behavior continues, go to Dr. Bowman

How code joins the code base

- **Getting a Change in**
 - Git-flow model - develop branch off of master, branch for features, merge features
- **Review Authority**
 - The person reviewing the change can stop it at any point when the request is in limbo?
 - If someone really believes a change needs to be stopped, they should explain their reasoning in detail with the group.
- **Author-Reviewer disagreements**
 - No more than 24 hours of deliberation
 - Reviewer and Author will be prepared to explain to the group why their solution is more correct

Hypothesis

- **Roles on a task**
 - **Responsible:**
 - **Accountable:**
 - **Consulted:**
 - **Informed:**
- **Way of working**
 -

- **Assigning Tasks**
 - A Team member will volunteer for tasks that fall within some of their experience, and the rest of the team will discuss the task assignment and either approve or disapprove.
- **Meetings (using structure described above)**
 - Sprint 1 meeting
 - Sprint 2 meeting
 - Sprint 3 meeting
- **Pair-Written Code**
 - Pairs will have a driver mainly writing code and an overseer being a second pair of eyes and source of information.

Open Disagreements

Branching and Versioning

- **Main Branch**
 - The main branch will always contain product ready code. Our team will use a hybrid Gitflow Continuous development method, this methodology has the benefit of stable releases and isolated features, but also the rapid development cycle of CD.
- **Time on a Branch**
 - The main branch will have a permanent lifespan and the dev branch will be active until merging into main is required. Feature branches will have short life spans, until the feature is finished.
- **Joining Main Branch**
 - Joining the main branch must only take place if the code is production ready. The main should always be runnable.
- **Version Numbers**
 - Version 1.0 will only be reached when the project is feature complete and has been tested to be so. Until 1.0 all releases will follow the structure of 0.x.y, where x is a meaningful update of features, and y is a patch for bug fixes of that particular release. The first release will start at 0.1.0